NewsBot Intelligence System 2.0

Technical Documentation

Author: Trilok Kalani | Course: ITAI 2373, Natural Language Processing | Group: SOLO
Builds on: ITAI2373-NewsBot-Midterm | Repository: github.com/Tikskalani/ITAI2373-Portfolio
Live demo (no install): tikskalani.github.io/ITAI2373-Portfolio

1. Overview

NewsBot 2.0 is a modular news-analysis platform. It takes raw article text and returns structured intelligence: category with a calibrated confidence, sentiment and emotion, named entities, an extractive summary, semantic-search matches, and discovered topics. A conversational layer maps plain-language questions onto those capabilities, and a Flask web application exposes the whole system through a browser.

The midterm delivered a single notebook that reached 97.5% test accuracy on BBC News. This final release refactors that pipeline into a `src/` Python package, adds four advanced modules (topic modeling with LDA and NMF, summarization and semantic search, multilingual analysis, and a conversational interface), wraps it in an automated test suite, and ships a web frontend.

2. System Architecture

The design principle is parse and vectorize once, then reuse. A thin facade, `NewsBot`, composes the analysis components; the four advanced modules extend it; and the Flask app is a presentation layer on top. The classifier owns the fitted TF-IDF vectorizer, while semantic search and topic modeling keep their own vectorizers because they are tuned for different goals (search favors recall, the classifier favors precision on the five known categories).

Figure 1. Layered architecture. Analysis and language components are reused by the conversational layer and the web app.

2.1 Module A - Advanced Content Analysis

- **classifier.py** - TF-IDF (unigrams + bigrams, sublinear_tf, tuned min_df/max_df) into Logistic Regression. Reports the winning category, its probability, the runner-up, and how many in-vocabulary terms it recognized. An uncertainty guard returns `uncertain` when the input has fewer than 2 recognized terms or top probability is below 0.35. compare_models benchmarks Naive Bayes, Logistic Regression, Linear SVM, and Random Forest with cross-validation.
- **sentiment_analyzer.py** - VADER compound plus TextBlob polarity and subjectivity, and a transparent 8-emotion lexicon.
- **ner_extractor.py** - spaCy en_core_web_sm entities, with an optional EntityRuler that corrects domain names such as Nvidia, OpenAI, and Manchester United.
- **topic_modeler.py** - LDA over counts or NMF over TF-IDF (English stopwords removed); exposes per-topic top words and a bar-chart visualization.

2.2 Module B - Language Understanding and Generation

- **summarizer.py** - extractive by default: sentences scored by mean TF-IDF salience, top n returned in order. An abstractive DistilBART path is lazy-loaded only on request.
- **embeddings.py** - SemanticSearch ranks a corpus by cosine similarity to a query; an optional SBERT backend swaps in dense vectors.
- **generator.py** - content enhancement into a readable brief, and WordNet query expansion.

2.3 Module C - Multilingual Intelligence

- **language_detector.py** - langdetect wrapper returning an ISO code.
- **translator.py** - deep-translator (Google backend) to a target language, default English.
- **cross_lingual_analyzer.py** - detect, translate to English, then run the standard English pipeline.

2.4 Module D - Conversational Interface

- **intent_classifier.py** - rule-based detection over six intents: classify, sentiment, entities, summarize, search, topics.
- **query_processor.py** - dispatches a query (and optional article) to the right component.
- **response_generator.py** - formats the result into natural language.

3. Representative Results

Classification accuracy from the midterm foundation, on a held-out BBC test split (five categories):

Model	CV accuracy	Test accuracy	Macro-F1
Linear SVM	0.972	0.975	0.975
Logistic Regression	0.968	0.970	0.969
Random Forest	0.951	0.948	0.947
Naive Bayes	0.945	0.943	0.942

Table 1. On live June 2026 articles the classifier labels tech 0.91, business 0.85, sport 0.96, and politics 0.97, and returns uncertain on short off-topic input rather than forcing a label.

4. Installation Guide

```
pip install -r requirements.txt
python -m spacy download en_core_web_sm

# library use
import sys; sys.path.append('.')
from src.newsbot import NewsBot
import pandas as pd
df = pd.read_csv('data/raw/newsbot_bbc.csv')
bot = NewsBot().train(df['text'], df['category'])
print(bot.analyze('Apple unveiled a new AI chip.'))

# web app
python app.py     # http://localhost:5000

# tests
pytest tests/ -q  # 8 passing
```

The base install runs on CPU with no API keys. Optional transformer summarization and SBERT embeddings are lazy-loaded and documented in requirements.txt.

5. Configuration Manual

Central configuration lives in `config/settings.py`. The most useful knobs:

Setting	Default	Effect
`MAX_FEATURES`	`5000`	TF-IDF vocabulary cap for classification
`N_TOPICS`	`5`	Number of LDA / NMF topics
`MIN_KNOWN`	`2`	Recognized terms required before a confident label
`MIN_CONFIDENCE`	`0.35`	Probability floor below which output is 'uncertain'
`summarizer method`	`extractive`	'abstractive' lazy-loads DistilBART
`search backend`	`tfidf`	'sbert' uses dense sentence-transformer vectors

6. API Reference (selected)

`NewsBot(use_domain_rules=False)`

- `train(texts, labels) -> self`
- `analyze(text) -> {classification, sentiment, entities}`

`NewsClassifier(max_features=5000, ngram_range=(1,2))`

- `fit(texts, labels, preprocess=True)`
- `predict(text) -> {category, confidence, runner_up, recognized_terms, key_terms}`
- `compare_models(texts, labels, cv=5)`

`TopicModeler(n_topics=5, method='lda')`

- `fit_transform(documents)`
- `get_topic_words(topic_id, n_words=10)`
- `get_all_topics(n_words=8)`
- `visualize_topics()`

`Summarizer(method='extractive')`

- `summarize(text, n_sentences=3)`
- `extractive(text, n_sentences=3)`

`SemanticSearch(backend='tfidf')`

- `index(documents) -> self`
- `search(query, k=5) -> list[(index, score, snippet)]`

`CrossLingualAnalyzer()`

- `to_english(text) -> {detected_language, translated_text}`

`QueryProcessor(bot, summarizer, search, topic_modeler)`

- `process(query, article=None) -> dict`

The full reference for every class and utility is in docs/api_reference.md.

7. Testing

`pytest tests/ -q` runs eight tests spanning preprocessing, classification, topic modeling, language models, the conversational layer, and an end-to-end integration test. The web app is covered by a Flask test client that asserts each route returns HTTP 200 with valid JSON. The seven tutorial notebooks in `notebooks/` also run end-to-end.

8. Known Limitations

- The small spaCy model makes occasional NER errors; en_core_web_trf would improve this.
- Lexicon sentiment misreads financial framing; a finance-tuned model such as FinBERT would be more reliable.
- Translation quality depends on the free Google backend and a network connection.
- The classifier's world knowledge is fixed to the training-corpus vocabulary, which is why the uncertainty guard exists.

AI-use disclosure: I used an AI assistant for code implementation, refactoring, debugging, and drafting. I chose the architecture and methods, ran and reviewed the code, verified outputs, and am responsible for the design and interpretation.